CSE 123A - Winter 2026

Group 8

Executive Summary

TODO

Contents

1 Introduction

1.1 Problem statement

Living with roommates can sometimes mean your filtered water device is empty when you need water. This can be very frustrating and leave you needing to drink tap water while other roommates get to drink cold refrigerated and filtered water.

1.2 Need Statement

We want a way to know if the container is out of water when we need water.

- Avoid having no water left in addition to everybody using the water filter, not knowing there is no water left.
- Need a way to know the current water level of the pitcher so that it isn't empty without everyone knowing it's empty.
- We need to know if the container is empty, in order to not run out of clean fresh water.

1.3 Goal Statement

- Keeping water level known at all times, while informing everybody in the household if the water level runs too low.
- Create a system to inform users if the container is empty.

If the container is empty, inform users.

1.4 Personas and Users

This is intended for shared households that rely on a shared filtration water container.

- College Roommates
  - John Doe, 20 years old, Student at UCSC. John lives in a dorm with two other roommates, and they all rely on a shared water filter for drinking water. Occasionally, John comes out to get water only to find that the water is empty. This creates frustration and delays as John has to choose between missing the bus to refill water or getting to class on time while being thirsty.
- Family
  - Jane Smith, 39 years old, working mother. Jane lives with her husband and two children. Jane gets home after a long, exhausting day of work and is feeling thirsty from the drive home. She goes to the shared water filter to find that it is completely empty. Jane feels frustrated with her stay-at-home husband and two children for not refilling the water filter. This causes her to lash out and yell at them until they all cry, creating a hostile environment in the household.

- Office
  - Joe Micheal, 30 years old, works with Excel spreadsheets. He has very limited time to refill his mug, and his boss, George, never refills the water filter. As a result, Joe's efficiency has substantially decreased at his desk.

1.5 Research into Existing Designs and Products

To understand where our design fits, we researched three existing products that partially meet the same need.

1.5.1 Withings Body Smart

The Withings Body Smart is a consumer Wi-Fi scale that measures weight, body fat, muscle mass, and water percentage using strain gauge load cells and bioelectrical impedance analysis. It syncs data automatically to the Withings smartphone app, where users can track trends and set goals. While it offers a polished user experience, the system is entirely closed—users must rely on Withings' proprietary app and cloud service, with no way to self-host data or customize the interface.

1.5.2 Wyze Scale Ultra

The Wyze Scale Ultra takes a similar approach but adds a 4.3-inch color display directly on the scale, so users can view 13 body composition metrics without needing their phone nearby. It supports Wi-Fi syncing to the Wyze app and integrates with third-party fitness platforms like Apple Health and Google Fit. Like the Withings, it is a closed system that cannot be modified or repurposed beyond its intended body composition use case.

1.5.3 Hackster.io ESP32 MQTT Weight System

On the DIY side, a Hackster.io project by The Embedded Things implements an ESP32-based weight measurement system using an HX711 amplifier and load cell. It features MQTT communication for real-time data streaming, multi-sample averaging for noise reduction, and automatic tare on startup. This project demonstrates strong signal processing but requires an external MQTT broker and separate dashboard software, adding complexity that our self-hosted approach avoids.

1.5.4 Where Our Design Fits

Our project fills a gap between these existing solutions. The commercial scales offer polished experiences but are closed systems tied to proprietary apps. The Hackster.io project is open-source and uses similar hardware, but depends on external services for data display. Our design combines the low cost and openness of a DIY build with a fully self-hosted web interface on the ESP32 that provides real-time weight display, historical logging, and alerts—all without needing external apps or cloud services.

1.6 Sustainability Statement

Our project promotes sustainable water consumption habits in shared households. Without knowing the current water level, users often open the fridge only to find an empty filter, leading them to either waste time waiting for it to refill or even use one-time plastic bottles. By providing real-time water level monitoring and low-level alerts, our system encourages refilling and reduces the likelihood of users turning to less sustainable alternatives. Over time, this helps households maintain consistent use of their reusable water filter rather than abandoning it out of frustration.

2 Design

2.1 Aesthetic Prototype

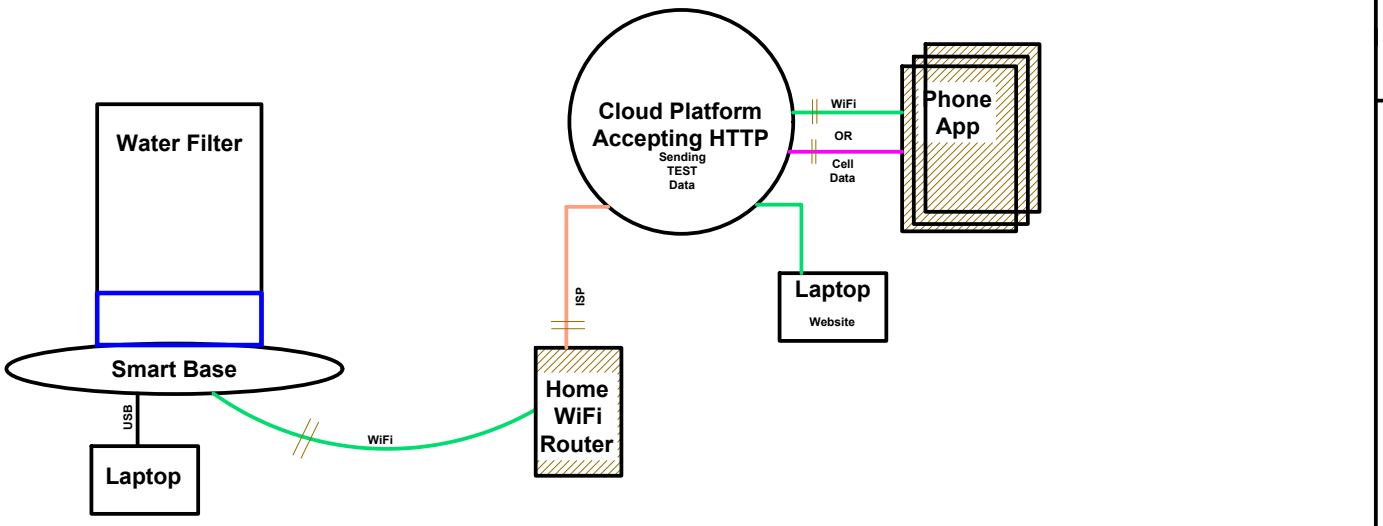
TODO

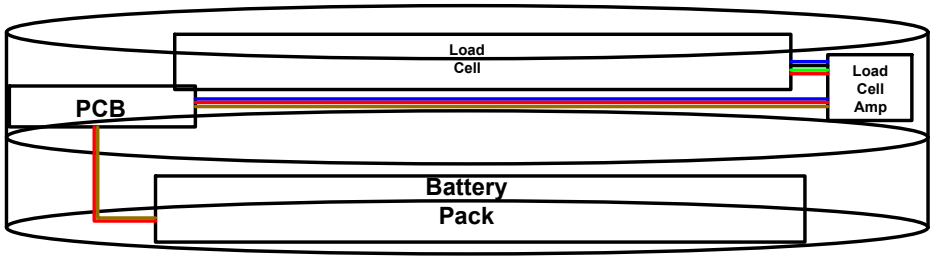
2.2 Design for Manufacture

TODO

2.3 Block Diagrams

The system block diagram represents the pieces of the device and how they are interconnected.





3D Smart Base
Housing

Initial Prototype System Block Diagram

Design Project

Network Provider
WiFi
Cell Data
Power
GND
ADC

2/27/26

Group 8

Filter-System.dwg

2.4 Wiring Diagrams

TODO

2.5 Web Application

The web application is the user-facing interface for the smart water-level monitor. The design goal is to provide a clear real-time estimate of remaining water in the container, allow user calibration for different pitcher sizes, and send refill notifications when the water level becomes low.

2.5.1 Initial Design

In its most simple form it must be able to display static data on a web page. We have created a .html file with this basic information, shown in the following figure.

Figure 1: Output of Initial HTML Code

From here there are a few changes that can be made. These changes follow the line of next steps to test this code and our designs.

- Dynamically updating data with a controller (Proof of concept testing w/ simple text)
- Sending and interpreting sensor data
- Graphic interface changes
- Sending push notifications to a phone

In order to test these things we will create a local server interface using one of our computers that hosts this data. Then we will use a microcontroller to send HTTP PUT requests. Finally, using the received data to change what gets displayed.

2.5.2 Current Design

The current design is a React application deployed on Vercel, with a Supabase backend for data storage. The frontend displays the latest water level reading as a percentage, along with a visual representation of the water level in the pitcher. Users can calibrate the system by setting empty and full reference points, which allows for accurate level estimation regardless of pitcher size or sensor placement. The application also includes functionality to send push notifications to users when the water level falls below a certain threshold.

2.5.3 System Architecture

The deployed architecture has three coordinated layers:

- **Frontend dashboard (React):** Displays the latest reading, computes estimated level percentage, and status indicators.
- **Vercel backend:** Receives sensor uploads and notification subscriptions.
- **Supabase Storage:** Stores water readings, calibration parameters, and browser push subscription tokens.

Sensor data is sent to the backend endpoint with an API key, validated, and inserted into persistent storage. The dashboard reads the newest value from Supabase on a polling interval and renders the latest state.

2.5.4 Frontend Design and User Experience

The current dashboard is intentionally minimal and task-focused. Core interface elements are:

- A large water-level visualization (pitcher graphic with animated fill height).
- A numerical percentage readout for quick interpretation.
- A water-present/empty status indicator.
- Metadata showing last update timestamp, measured weight, and battery voltage (when available).

Users can quickly assess whether refill action is needed without interpreting raw sensor values.

2.5.5 Calibration

To support different pitcher geometries and sensor offsets, the application includes user calibration controls:

- **Calibrate empty:** Stores the current reading as the empty reference.
- **Calibrate full:** Stores the current reading as the full reference.
- **Reset calibration:** Clears stored values and falls back to default constants.

The level estimate is computed by linearly mapping measured weight between empty and full references, then clamping the result between 0% and 100%. This makes the UI behavior predictable while still allowing practical adjustments.

2.5.6 API Endpoints

The application has two primary API endpoints:

- **POST /api/ingest:** Receives sensor readings (device ID, weight, battery) from the microcontroller and stores the data in Supabase. If the weight is below a configured threshold, triggers a low-water notification to registered users.
- **POST /api/register-token:** Registers browser push subscriptions from the frontend and saves them in the database.

2.5.7 Push Notifications

The push notification feature is designed as an event-driven subsystem that alerts users only when refill action is needed. The implementation uses standard Web Push protocols with VAPID authentication and is integrated into the ingest API used for sensor data.

1) Subscription Registration When the dashboard is loaded, the browser requests notification permission. If permission is granted, a service worker is registered and a Push API subscription is created (or reused if it already exists). The subscription object is sent to the backend endpoint and stored in the `notification_tokens` table in Supabase.
This registration path has two design goals:

- maintain a durable list of reachable user devices, and
- avoid duplicate subscriptions by upserting on token value.

2) Alert Trigger Condition Sensor readings are received by the ingestion endpoint and stored in `water_readings` table. For each new reading, the backend computes current and previous water-level percentages and checks for a low-level threshold crossing. The low-water threshold is currently set to 20%. This design prevents repeated notifications while the level remains continuously low.

3) Delivery If the trigger condition is true, the backend loads all stored subscriptions and sends a Web Push payload to each valid endpoint. The payload includes a:

- notification title
- notification body
- structured data (event type and level percentage) for client-side handling.

The service worker receives the push event, displays the notification, and handles click interaction by focusing an existing app tab or opening the web page.

2.6 HX711

This file will show the necessary components to be able to collect/read data from the HX711 sensor to your microcontroller. The goal is collect sensor readings, convert the readings to grams.

2.6.1 Important Wires

SCK is the clock line. DOUT is the data line. This means whenever DOUT is low it is ready to read. The clock line comes in pulses.

2.6.2 Read Data

The HX711 Sensor is a 24-bit analog to digital converter. To read data, you will need to be able to read the raw 24bits. The wire that matters here is SCK, since SCK is a pulse that will shift 1 bit each time, you need to do this 24 times for the full number. Additionally, there are extra pulses needed for the HX711 to select Chanel. This is inferred as my define PULSES_TOTAL 25. Create a delay when waiting for DOUT to go low again, sometimes it might stay high.

2.6.3 Average Sampling

Sometimes, the sensor will spike its readings. Average sampling allows us to deal with outliers.

2.6.4 Calibration

The Sensor gives raw data, as counts. From testing, have a hard coded number that when using to calculate the grams from the raw data, just divide raw by the hard coded number.

2.6.5 Debug

For ESP32, the tool has ESP_logi to help out when monitoring for example.

2.7 Testing HX711

To test the code written for this project, we defrost had to create a simulated version of the sensor. Regardless of what the sensor reports we want to make sure that all of our math and interpretation of incoming data is correct. To do this, the functions were adapted to either take input from GPIO pins, or from a specific test file.

2.7.1 Test File

Inside the test file are adapted versions of each function used by the main HX711 file. These include the following:

- hx711_set_sck

- hx711_get_dout
- read_raw
- get_raw_weight
- tare
- get_grams

With each of these functions, the test file simulates reading data from the sensor.

2.7.2 Simulated Data

Using some set values defined at the top of the file including a hardcoded offset and scaling factor, the test file first sets the weight and tares it. Then based on a random number between 0 and 2, it sends either a 0, small weight, or large weight value.

When the test file calls to the read weight functions, some small noise of $\pm 1g$ is used simulate the variability of the sensor reading. The sensor sends data in 24 bit samples, so the test code also includes the 24 bit shifting. Once the simulated data is created, the main function compares it to the expected result. If it's within one gram of the expected low high or zero value, then the value passes. This tolerance is arbitrary for the testing and will be different for the production code.

2.8 State Transition Diagrams

TODO

2.9 Technology

TODO

2.10 Simulations

TODO

3 Evaluation

3.1 Functional Prototype

TODO

3.2 Testing

TODO

A Problem Formulation

A.1 Conceptualisations

TODO

A.2 Brainstorming

TODO

A.3 Decision Table

Items to decide on:

- Microcontroller
- Sensor
  - Ultrasonic: Pointed down from the lid onto a floating object
  - Load Cell: Place the container on the load cell to measure the weight and calculate the water level
  - Laser: Shoots a laser into the water from the lid, and the distance is sent back to the sensor
  - Camera: Setup to watch the water filter and measure the water level based on computer vision
  - Non-Contact Sensor: Patch that sits on the side, when water level goes below sensor it can alert the device
- Server
  - HTTP
  - AWS
  - Vercel
- Challenges
  - Signal in the fridge
  - Size limit
  - Contact Vs No Contact
  - Wifi connectivity
  - Power source (battery, USB, etc.)

Based on the decision matrix, the weight sensor is the best sensing method because it has the highest weighted total score (4.55). It performs especially well in measurement quality while still keeping cost, power, and overall complexity at a strong level compared to the other options.

Criteria (Weight)	**Capacitive**	**Weight**	**Laser**	**Ultrasonic**
Cost (0.30)	5	5	3	5
Power (0.20)	5	4	3	4
Complexity (0.25)	3	4	2	3
Measurement Data (0.25)	3	5	3	3
Weighted Total	**4**	**4.55**	**2.70**	**3.85**

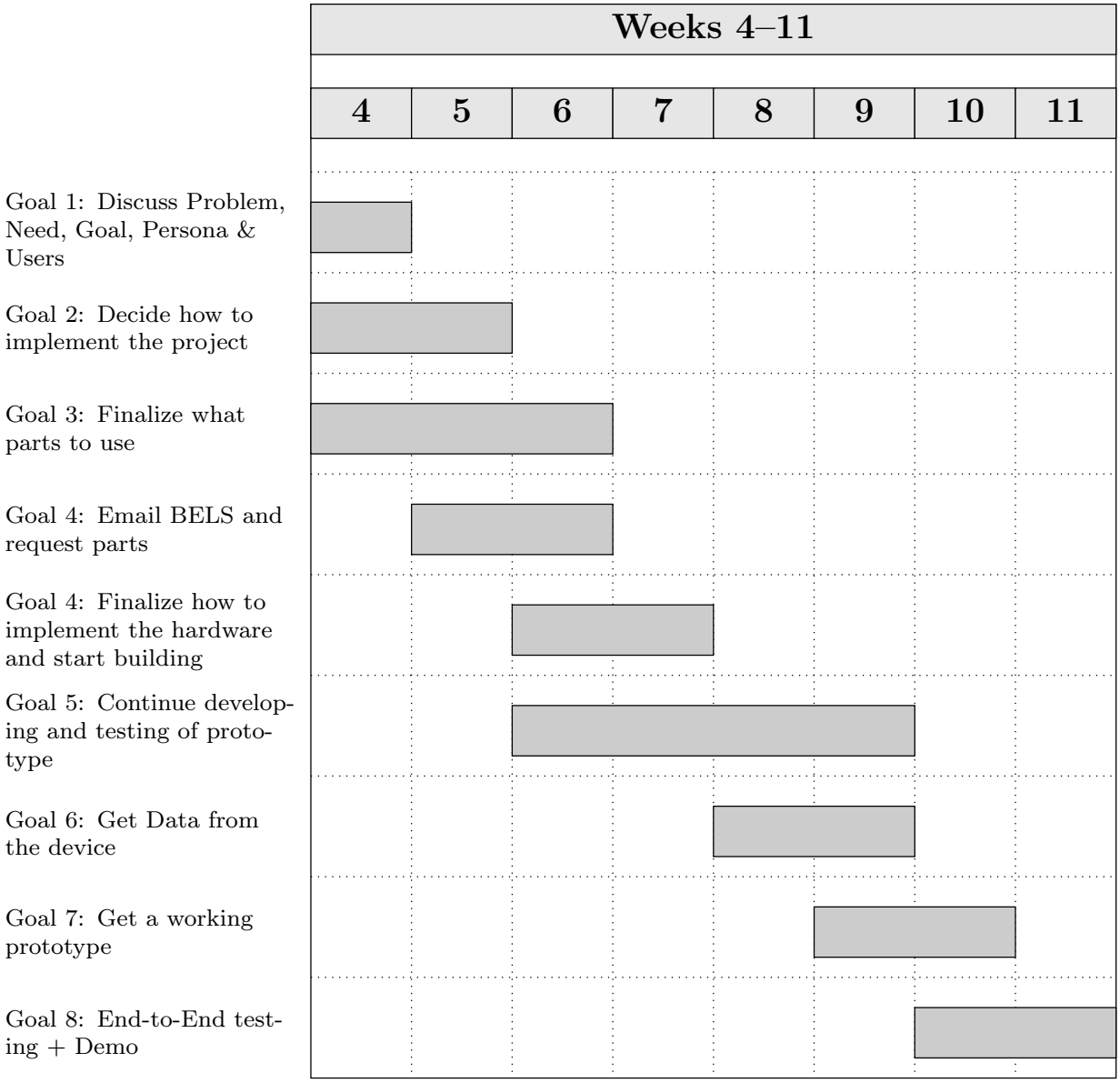
Table 1: Decision matrix comparing feasible sensing approaches (1 = poor, 5 = excellent)

A.4 Morphological Charts

TODO

B Planning

B.1 Basic Plan / Gantt Chart



B.2 Division of Labor

TODO

B.3 Collaboration

TODO

C Test Plan & Results

C.1 Test Plan & Results

There are multiple sections of our design that require testing individually, then together as a whole. The following list represents the pathway we will take when testing our prototype segments.

- ESP Data Reporting
- ESP Web Posting
- Vercel Receiving and Displaying Data
- Notification System
- Phone App Integration
- Prototype Setup Out-Of-The-Box

ESP Data Reporting

The pressure sensors we use to build the base will output data to our ESP device. We want to be sure the ESP is capable of receiving this data and displaying it to the debug terminal, which will be IDF MONITOR. Once we can verify posting to the terminal we will move to the next stage of the data reporting testing.

With data being displayed properly, we can now use this to interpret what an empty versus full pitcher could be expected to weigh. The first test will include to test a known weight to make sure we get accuracate weight readings. Next we will ensure the that when there is no wight on the plate, the data will be read as 0 g. Once this correct, we will move on to the more complex calibration with Pitcher + water.

We will use different types of containers given the variety of containers possible to be used by customers. We will test at different set water levels, and verify that the expected weight represents the following formula.

$$W_{total} = W_{container} + V * \frac{g}{mL} \tag{1}$$

Water weighs 1 gram per milliliter so if we keep track of the volume we put into the container we know exactly what to expect for output weight if we know the base weight of the container alone.

ESP Web Posting

To test this functionality the simplest way is to have the ESP send some sort of HTTP POST pointed at one of our local machines. We can receive and print this data and then move forward to testing sensor data output.

With a working sensor we can trigger a POST request when an event happens. In this test case it could be picking up the container from the base which triggers a POST request

with some set information. Upon successful receipt of this we can confirm that the HTTP method is working on a small scale and move to the next segment of testing.

Vercel Receiving and Displaying Data

Once we have confirmed the ESP can send POST requests to a local machine, the next stage is to implement that using Vercel.

Notification System

With data being sent out of the ESP, the next stage is to test if we can send a notification on some set event. For a test case this could be a container being placed on the device. The data sent as part of this notification could be a value representing the weight of the container, or an estimate of how much liquid is inside.

If we know how much liquid is inside the container when we place it on the base, it is easy to verify if the value we receive in the notification is the correct volume. Using different volumes we can create a table to show how close the reported volume was, and tweak our math to get it closer to accurate with different containers.

When we pass the fake data tests, we will compare the debug terminal and the notification water levels to ensure that it is correct.

Phone App Integration

Prototype Setup Out-Of-The-Box

In order to set up the prototype, the web app must connect to the ESP through its hosted soft AP and scan a QR code to enter the WiFi credentials. To test this, we will try to connect to the ESP on boot multiple times using different devices, such as iOS vs Android, and see if it is still able to connect through soft AP. We will also test different types of WiFi networks, such as mobile hotspots, traditional password-protected networks, and networks with outside authorization such as university WiFi.

The user must also calibrate the device by measuring the empty and full weights of the filter on the weight sensor. On the webapp, they will be given instructions step-by-step to do this. In order to test this process, we will give the device to a new user and see if they are able to clearly follow the instructions and calibrate the device correctly. We will also test robustness by intentionally disobeying the instructions, such as by weighing it empty both times, to see our device handle it.

D Review

D.1 Team Reflections

TODO